

How to use Claude Code

/context

It is the memory that Claude holds while working with us on a project. Before starting any prompt it answers according to the context it has already stored so it is important. But the context has limited memory so if it fills up, Claude will compact the context and remove older things it remembered. Therefore subagents are helpful if our subagent does the same task that we have prompted then the subagent opens a new smaller context and just returns the summary of the task to the main context and deletes itself not wasting context memory.

[CLAUDE.md](#)

It stores persistent memory about the project. Each time we open Claude code in folder, without [Claude.md](#) Claude would read all the files to understand. Therefore we have a [Claude.md](#) and we can also put some rules in it that we want it to follow always. There are two types of [Claude.md](#) one is global (~/.[CLAUDE.md](#)) and one for each project (project/.[CLAUDE.md](#)). Global one will apply for all projects we do. Local ones for our project and local supersedes global for our project.

Subagents

As already mentioned these are useful for saving context memory. These are .md files inside a folder (agents) which is inside .claude folder.

```
.claude
  agents
    agent1.md
```

Claude automatically uses agents if you have specified it for the task that you prompted no need of explicitly telling which folder to use.

There are also built in subagents like explore, plan, etc which do their respective tasks.

Commands

These are .md files that tell Claude how to do something once. These are mainly helpful so that we do not need to write a very long prompt. Our file will define what Claude needs to do. These need to be explicitly specified in our prompt for Claude to use it unlike subagents. File Structure is similar to agents

```
.claude
  commands
    command1.md
```

To use it [/command1.md](#) in your prompt.

Skills

They are very much similar to commands but are more bigger and detailed than commands and may use some tools also.

(When I used Claude code it told me skills will also come under commands folder)

It is also a .md file.

Model Context Protocol (MCP)

This helps to connect claude code with external tools But be careful too many tools eat up the context memory too fast therefore use only the tools needed.

Hooks

These are deterministic in nature. They always run there a few things in this like pre tool use and post tool use means the hook should run before the tool is used or after.

This is inside .claude , inside the folder settings.json

We have to update the settings.json file itself for our own hooks.